

Contest 1: Language Modeling — A Predictive Keyboard

T6523

Branch: new_data

Abstract—We build a predictive-keyboard system that, given a left context and the first letter of the next word, predicts the exact word. Work proceeded in two phases. Phase 1 was carried out against a training/development split that was later found not to be the official contest data; all Phase-1 numbers are retained here as an architecture and ablation study but are not the basis for the submitted system. Phase 2 began once the correct files (`train_final.src.tok`, `dev_set_final.csv`, `test_set_no_answer_final.csv`) were obtained on the `new_data` branch, and is the authoritative pipeline. The final system routes each prediction by answer category — a deterministic rule for punctuation, a Kneser–Ney 5-gram trained on a number-augmented corpus for anonymized numbers, and a three-way probability blend of a fine-tuned Qwen2.5-3B, a zero-shot Mistral-7B, and a Kneser–Ney 5-gram for ordinary words — reaching 77.68% overall / 75.09% word-category accuracy on a held-out, pooled 5-fold cross-validated sample of the (correct) development set, up from a 55.51% word-accuracy trigram-family baseline. The final, full-scale run (§III-B6) — tuned on one stratified half of a 20,000-row dev subsample and reported on the other half ($n = 9,999$) — lands at 78.60% overall / 76.09% word accuracy (± 0.8 pt at 95% CI), and produced the submitted `test_set_pred.txt` (94,826 predictions).

I. INTRODUCTION

The task (per the contest specification) is next-word prediction for a simulated predictive keyboard: given a left context and the first letter of the target word, output the single most likely completion — not a full probability distribution over the vocabulary. At minimum, two of {trigram, 4-gram, 5-gram, GPT/GPT-2-style neural LM} baselines are required; we implement all four baseline families plus two fine-tuned/zero-shot large pretrained models, and combine the strongest of them into a single blended system.

Timeline note. The project was first built end-to-end (data exploration, n-gram training, a from-scratch GPT-2, a KenLM-based pipeline, and a Qwen2.5-0.5B LoRA fine-tune) against a training/dev split that turned out to be incorrect — not the data the contest actually provides. That entire body of work is summarized in §III-A/§IV-A as **Phase 1**; it remains useful as a controlled study of modeling choices (smoothing, interpolation, n-gram order, architecture comparisons) but its absolute accuracy numbers do not apply to the real task. Once the correct data was located (the `new_data` branch: `train_final.src.tok`, `dev_set_final.csv`, `test_set_no_answer_final.csv`), the project was rebuilt from EDA up — **Phase 2**, §III-B/§IV-B — and it is Phase 2 that was tuned, validated, and submitted.

II. DATA

A. Official data (Phase 2, authoritative)

TABLE I
OFFICIAL CONTEST DATA FILES

File	Notes
<code>train_final.src.tok</code>	3,803,957 lines / 126.7M tokens / 99,018-word vocab; one sentence per line, lower-cased, Penn-Treebank-style tokenization
<code>dev_set_final.csv</code>	94,488 rows: context, first letter, answer
<code>test_set_no_answer_final.csv</code>	94,826 rows: context, first letter (no answer)
<code>gigaword.tar.gz</code> (optional)	149 monthly files, 1994–2006, 2.4GB compressed; NYT Gigaword, supplementary corpus (§III-BE)

Matches the contest’s stated “~128 million tokens” (we measure 126.7M) and the Penn-Treebank tokenization convention (i ’m going to the store - lrb - tomorrow - rrb -). No PTB placeholders (-lrb-, -rsb-, etc.) appear anywhere except the literal parenthesis substitution.

Answer categories (regex: all-digit $\rightarrow$ *number*, $[a-z']^+$ $\rightarrow$ *word*, else *symbol*), measured on the 94,488-row dev set: 83,228 word / 9,237 symbol / 2,023 number.

- **symbol** rows are near-fully determined by first letter alone (99.6% majority-vote ceiling; 24/27 letters are fully deterministic in dev).
- **number** rows are digit-length placeholders, not real values (every number is anonymized to a run of 1s: 1234 $\rightarrow$ 1111, lengths 1–8). No OOV risk, but first-letter alone is a weak signal (39.6% majority-length guess) since every number placeholder starts with 1.
- **word** rows are ordinary open-class vocabulary and are where nearly all of the modeling effort goes.

Known structural gap: the training tokenizer folds every literal `[/]` into `[UNK]` (414,394 occurrences), so the model can never emit a literal bracket, while dev/test retain 124 literal-bracket answers (0.13% of dev). This is a fixed ceiling loss, not a coverage problem to fix with more data.

Vocab coverage is tail-heavy: 90% of train’s token mass is covered by ~4,400 words (4.4% of the 99,018-word vocabulary); 99% needs ~30,800 words. Dropping the rarest 20–70% of vocabulary by frequency costs comparatively little dev OOV (0.19% at 20% dropped, 1.06% at 70% dropped) — pruning

rare words is cheap, which motivated the pruning experiments in §III-BA.

Context/answer lengths are short and thin-tailed (dev context median 24 tokens, p99 42; answer median 4 characters, p99 12), so context length is never the modeling bottleneck — order-4/5 n-grams and a few hundred tokens of transformer context are both comfortably sufficient.

B. Phase-1 (incorrect) data, for reference only

Phase 1 used `data/train.src.tok` (raw, unclean, ~703MB) and a masked NYT Gigaword corpus `data/gigaword.tok` (~7GB), with dev/test files `devv_eval.csv/devv_test.csv`. These are a different, earlier snapshot from the official contest files and are not used for the submitted system; they are retained only insofar as the experiments run on them (architecture comparisons, smoothing choices) inform Phase-2 design decisions (§III-A).

III. MODEL

We discuss every model attempted, in the order it was tried, split by phase.

A. Phase 1 — preliminary experiments (data later found incorrect)

This phase established the pipeline architecture and answered several modeling questions that transferred directly to Phase 2, even though its absolute accuracy numbers do not apply to the real dev/test split.

- **KenLM 5-gram pipeline** (Model A: in-domain corpus; Model B: masked NYT Gigaword), built with `lmplz/build_binary`, modified Kneser-Ney smoothing.
- **Ranking-logic ablation:** ranking candidates by `BaseScore` `argmax` vs. n-gram-length-first tie-break changed accuracy by only ~0.1–0.3pp — KenLM’s own backoff already orders candidates similarly either way.
- **Corpus-cleaning ablation:** stripping punctuation from the training corpus cost –6.2pp because the eval contexts still contain punctuation tokens; the raw, unclean corpus is the *correct* choice, not a shortcut.
- **The real bug:** a missing deterministic symbol-route rule was silently failing ~10.7% of rows; wiring it in recovered +10.7pp overall accuracy for free — a reminder that a missing routing rule can dwarf any modeling change.
- **Linear interpolation** of Model A + Model B, $P = \lambda \cdot 10^{\log_{10} P_A} + (1 - \lambda) \cdot 10^{\log_{10} P_B}$, λ grid-searched on perplexity, beat either model alone by ~20% perplexity reduction and +2.5pp accuracy, confirming that even a much smaller in-domain corpus and a much larger out-of-domain corpus carry complementary signal worth blending — the core idea reused in Phase 2’s 3-way blend.
- **Order-aware interpolation** (letting whichever model matched the deeper n-gram order win, instead of a fixed λ) was tried and was *worse* by 0.5pp: n-gram order is

not a reliable cross-model quality signal, since an out-of-domain model can match a deep order on a generic phrase with weak actual probability mass. Probability-space blending, which naturally weighs by mass, wins.

- **N-gram order search** (4 → 7, `KENLM_MAX_ORDER` rebuilt from the stock cap of 6): monotonic but sharply diminishing gains (order 5→6 +0.1pp, 6→7 +0.07pp) — not worth pushing further.
- **A from-scratch GPT-2-style transformer** (497M params, word-level vocab, trained from random initialization, masked to the first-letter bucket at every position) beat the best n-gram blend by +7–8pp, showing that a neural LM’s smoother generalization over sparse contexts clearly outperforms count-based backoff once enough compute is available.
- **A Qwen2.5-0.5B LoRA fine-tune** underperformed *every other approach*, including the n-gram (–20pp vs. n-gram, –28pp vs. the from-scratch GPT-2). Root-caused to severe undertraining (an 8-hour compute budget covered only 0.77 epochs of the corpus, loss still falling with no plateau) rather than a scoring bug — the errors were fluent but off-corpus completions (`kilometers`→`km`), consistent with pretrained general-web priors still dominating a barely-adapted LoRA adapter. This motivated Phase 2’s choice of a much larger base model (Qwen2.5-3B) and a longer, resumable fine-tuning schedule.
- **A 3-model perplexity-space ensemble** (n-gram A + n-gram B + GPT-2) further reduced perplexity by ~1%, reinforcing that blending diverse models keeps paying off even after one model dominates solo accuracy.

B. Phase 2 — corrected data (authoritative pipeline)

1) *N-gram baselines (contest-required minimum):* A pure-Python order-4 count-table model (`scripts/train_ngram.py`), `<s>/</s>` padded, pruned at `-min-count 10` on order ≥ 2 (context, word) pairs, with count-preserving fallback when every continuation of a context is rare. Order 4 was chosen over 5 because sparsity is already severe at order 4 (98% of 4-gram contexts already fall into the “everything is rare” fallback case) — a 5-gram would gain little while costing more RAM/time. Trained twice:

- **Model A** on `train_final.src.tok` directly (684MB model).
- **Model B** on NYT Gigaword, masked onto Model A’s vocabulary. Training this at scale required patching the trainer to prune *during* counting (`-prune-every-lines`) rather than after, since a one-shot end-filter’s peak (pre-pruning) memory exceeded a 14GB cap and crashed the first attempt.

Inference (`scripts/infer_ngram.py`) backs off order 4→3→2→1, filtered to candidates starting with the required letter, `argmax` by count; the symbol category is predicted trivially as the given first letter (already ~99.6% optimal per the EDA, so nothing is gained by routing it through the n-gram).

2) *Pretrained LLM shortlist (contest’s neural-LM baseline)*: Rather than assume a benchmark leaderboard transfers to this task, eight candidate base models were measured **zero-shot, no fine-tuning**, on the actual task (a 2000-row seed-42 sample of dev, first-letter-constrained greedy decoding) instead of by proxy benchmarks (HellaSwag/ARC/MMLU). **Qwen2.5-3B** won (70.26% word accuracy, beating the initially benchmark-favored Mistral-7B-v0.1 at 69.92% while being under half the parameter count and $\sim 1.7\times$ faster) and was kept as the base model for fine-tuning. A later check against the newer Qwen3.5 family (released after the original shortlist) confirmed Qwen2.5-3B still wins on word accuracy in this environment — bigger/newer did not mean better here.

Every zero-shot model scored near-0% on the *number* category: no LLM naturally generates a repeated-digit placeholder string like 1111, since it isn’t real text. This is not a capability gap — a digit-only first letter is an unambiguous “this is a number” signal, so *number* is routed around the LLM entirely (§III-BE), the same way *symbol* is handled deterministically.

3) *LoRA fine-tuning*: Qwen2.5-3B was LoRA-fine-tuned ($r=16$, $\alpha=32$ on `q/k/v/o_proj`) on `train_final.src.tok` as a plain packed causal-LM objective — no context+“[letter]” $\rightarrow$ target prompt format; the first-letter constraint is applied purely at inference time as a logit mask, keeping training data identical to raw text. Two Kaggle T4 sessions ($\sim 6h + \sim 2h$) were followed by a further **6-hour continuation** run later in the project once a fixed candidate-generation ceiling was diagnosed (§III-BE). Evaluated by beam search ($\text{num_beams}=k$), letter-masked at the first decoding step, top- k distinct words after de-duplicating beams.

An error analysis on the first checkpoint’s remaining top-1 mistakes found: **100%** of wrong predictions still start with the correct letter (the mask works); **47%** are recoverable by widening to top-5 (a ranking error, not a missing candidate); and the dominant failure mode is genuine function-word ambiguity unknowable from left-context alone (a/an, that/to, and/at) rather than a model deficiency.

Two rerankers were tried on top of the beam and **both hurt accuracy**: a frozen MiniLM cosine-similarity reranker (-11.4pt — topical/retrieval similarity is the wrong signal for grammaticality) and a trained MiniLM listwise cross-entropy reranker (-7.4pt vs. baseline — it only ever sees the same context the LM already conditioned on, so it has no independent signal to add). Both were abandoned.

4) *N-gram-LLM blending*:

$$\text{score}(w) = \lambda \cdot \log P_{\text{LM}}(w) + (1 - \lambda) \cdot \log P_{\text{ngram}}(w) \quad (1)$$

over the union of the fine-tuned LM’s top-5 beam candidates and the n-gram’s own best guess(es) for that first letter — so the blend can recover a word the LM’s beam never generated, not merely re-rank a fixed list. λ is tuned by grid search. An initial single 80/20-split estimate (72.50% word) was later found to be small-sample noise ($n=1640$, $\pm 2\text{pt}$ at 95% CI); the trustworthy number comes from **pooled stratified 5-fold**

cross-validation, where every row receives exactly one out-of-fold prediction: **71.52% word / 74.47% overall**.

Two refinements to the n-gram side of the blend were tested under the identical CV harness:

- 1) **Swap the raw-count n-gram scorer for a KenLM modified-Kneser-Ney 5-gram**. $+0.22\text{pt}$ word, $+0.19\text{pt}$ overall — real but modest; the blend’s ceiling is set more by the candidate set than by how well-calibrated either side’s probabilities are.
- 2) **Teacher-forced full-word scoring** instead of trusting the beam’s own sequence log-probability, over the widened union set ($\text{LM top-5} \cup \text{n-gram top-10}$). This removes beam-search pruning distortion on multi-subword words. $+0.88\text{pt}$ word, $+0.76\text{pt}$ overall — the largest single gain of this stage.

A **learned blender** (gradient-boosted trees over 7 features: LM log-prob and rank, KN5 log-prob, n-gram backoff order, unigram frequency, word length, is-function-word) was tried in place of the scalar λ and came out flat-to-slightly-worse (72.55% vs. 72.62% word) — the extra features correlate too heavily with what a single λ blend already captures, echoing the reranker lesson above: added model complexity does not help when the “new” signal isn’t actually independent.

5) *Second LLM and number-specific model*: **Mistral-7B-v0.1, zero-shot**, was added as a third blend member — teacher-forced over the exact same candidate set already fixed by Qwen (no new candidate generation, no beam search for Mistral). It was chosen for architectural diversity (different pretraining corpus, different tokenizer) rather than solo accuracy: two more diverse architectures were screened first (state-space mamba-2.8b: 65.55% zero-shot word accuracy; pure-RNN RWKV-4-3B: 60.66%) but both trailed Qwen by enough to make a net blend gain unlikely, whereas Mistral’s -0.34pt solo gap was a safer bet:

$$\text{score}(w) = \lambda_q \text{qwen_tf}(w) + \lambda_m \text{mistral_tf}(w) + (1 - \lambda_q - \lambda_m) \text{kn5}(w) \quad (2)$$

grid-searched over the (λ_q, λ_m) simplex, KN5 scores pre-computed once per row so the 2-D grid search runs in under two minutes instead of the ~ 37 hours a naïve re-scoring grid would take. Result: $+2.02\text{pt}$ word, $+1.76\text{pt}$ overall — the single largest gain of the project. Tuned weights land at $\lambda_q \approx 0.20-0.30$, $\lambda_m \approx 0.50-0.60$: **Mistral is weighted more heavily than Qwen** despite scoring slightly lower solo, direct evidence that its errors are decorrelated enough from Qwen’s for the blend to lean on it harder than raw solo accuracy would predict.

Number-specific KenLM. The general n-gram routes the *number* category, but it is trained on the same corpus as everything else — no information specific to numbers. A second KenLM 5-gram was trained only on number-containing lines, pooling `train_final.src.tok`’s own number lines with a matching slice of NYT Gigaword (anonymized to the same digit-length placeholder scheme before the vocabulary lookup, so a literal “37” correctly becomes the in-vocabulary

token "11" rather than falling to [UNK]). This improved number accuracy from 71.97% to **74.59%** (+2.6pt) on the full number subset. A same-corpus bag-of-words classifier was also tried for this category and scored *worse* (48–54%) than the general n-gram — a model “specific to” a category only helps if it draws on genuinely new data, not just a narrower view of the same data.

Continued fine-tuning + wider beam. The LoRA adapter was resumed for 6 more hours on a fresh corpus chunk, and beam width was widened from $k=5$ to $k=10$ (an error analysis had found 9.5% of word rows hit a hard candidate-set ceiling at $k=5$). Solo greedy accuracy jumped 69.72%→73.15% word (full 8198-row sample), and the candidate ceiling rose from top-5 84.0% to top-10 88.1%. Re-running the full 3-way blend on top of this: **+0.45pt word**, **+0.39pt overall**, with λ_q now *outweighing* λ_m (0.50 vs. 0.40) — the retrained checkpoint’s own accuracy gain is large enough that the blend leans back toward Qwen rather than only on Mistral’s diversity.

6) *Final full-scale run and submission:* Everything in §III-BD-E was validated on a fixed 9,448-row pooled-CV sample. For the real submission, Stage 1 (Qwen scoring/generation) and Stage 2 (Mistral scoring) had to be run at full scale: 94,826 test rows (mandatory, run in full) plus enough dev rows to tune (λ_q, λ_m) honestly. Since (λ_q, λ_m) is only two scalars, dev was **stratified-subsampled 94,488 → 20,000 rows** (seed 123, stratified by answer category) rather than run in full — a 20k sample gives ± 0.6 pt at 95% CI on the tuned number, more than enough precision for two parameters, at roughly half the compute of the full dev set.

Compute split. Stage 1 (Qwen) was split by row range across hardware and run concurrently: the local RTX 4060 took rows [0.0, 0.207) (4,140 dev + 19,628 test rows), a single-GPU Kaggle T4 session took the remainder — split point chosen from each device’s measured solo throughput so both finished around the same time. Dual-GPU Mistral (two T4s, separate processes) scaled cleanly to ~ 7 –11 rows/s combined; dual-GPU Qwen did not — the second process was repeatedly host-RAM SIGKILLED regardless of LoRA-vs-merged loading, so Qwen’s Kaggle leg ran single-GPU. In measured full-scale throughput, Kaggle’s larger slice became the long pole despite its higher per-row rate, since the local/Kaggle split ratio had been tuned from an earlier smoke test that undershot at full scale. The two hardware slices were merged back by absolute row index (contiguity-checked) before Stage 2.

Two bugs were caught before they reached the submitted file, not after: an over-broad error-grep in the orchestrator script false-flagged a successful Stage 1 run as failed (row counts confirmed it hadn’t failed); and the Mistral notebook still carried a leftover smoke-test `-limit 100` from earlier debugging, silently under-running Stage 2 on the first pass — caught immediately by the row-count assertion built into `scripts/combine_final.py`, not by any external monitoring. Both were fixed and Stage 2 was re-run and re-verified before the blend was trusted.

Final tuned weights and result. (λ_q, λ_m) were grid-searched on one stratified half of the 20k dev subsam-

ple ($n=10,001$) and evaluated on the other, untouched half ($n=9,999$):

TABLE II
FINAL TUNED BLEND WEIGHTS

Weight	Value
λ_{qwen}	0.40
λ_{mistral}	0.50
$\lambda_{\text{kn5 (remainder)}}$	0.10

TABLE III
FINAL HELD-OUT RESULT, REPORT HALF ($n=9,999$)

Category	Accuracy	n
word	76.09% (6602/8676)	8676
symbol	99.10% (1099/1109)	1109
number	73.83% (158/214)	214
overall	78.60%	9,999

At $n = 9,999$ and $\hat{p} = 0.786$, a normal-approximation 95% CI on the overall figure is $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n} \approx 78.60\% \pm 0.80\text{pt}$ — consistent with the $\pm 0.6\text{pt}/20\text{k}$ -row planning estimate used to size the subsample. This half of the dev subsample was never used to tune λ , so 78.60% is a genuine held-out estimate, and — being drawn from a different (later, larger, stratified) sample than the 9,448-row pooled-CV set in §III-BD-E — is not directly comparable point-for-point to the 77.68% progression figure, though both tell the same story and land within a point of each other.

`scripts/combine_final.py` merged all stages and wrote `weights/test_set_pred.txt`: 94,826 predictions, one per test row, 0 blank lines — the submitted file.

IV. RESULTS

A. *Phase 1 (incorrect data — not used for grading, shown for model comparison only)*

B. *Phase 2 (official data — authoritative)*

All word-category numbers in Table VI–VII are out-of-fold predictions from pooled 5-fold CV — every row is scored by a blend tuned on the other four folds, so 77.68% is a genuine held-out estimate, not a number read off the same split it was tuned on.

C. *Final full-scale result (submitted, §III-B6)*

Held out on the un-tuned half of a stratified 20,000-row dev subsample; test-set predictions (94,826 rows, `weights/test_set_pred.txt`) were produced with these same weights on the full test set. This is the number the submitted system actually achieves and supersedes the earlier 9,448-row pooled-CV estimate (§IV-B) as the project’s final reported result — the two agree to within about a point.

TABLE IV
PHASE-1 MODEL COMPARISON (INVALID DATASET, ARCHITECTURE COMPARISON ONLY)

Model	Word/alnum (eval / test)	Symbol	Notes
N-gram, best interpolated blend ($\lambda=0.71$, mask-fix)	60.29% / 60.15%	$\sim 99.4\%$	KenLM A+B
From-scratch GPT-2 (497M, word-level)	67.44% / 68.09%	$\sim 99.4\%$	random init, first-letter masked
Qwen2.5-0.5B LoRA (8h, 0.77 epochs)	39.83% / 39.28%	$\sim 99.4\%$	severely undertrained

TABLE V
N-GRAM BASELINE, MODEL A, ORDER 4, DEV SET

Category	Accuracy
word	55.51%
symbol	99.13%
number	71.68%
overall	60.12%

V. DISCUSSION: WHAT MATTERED AND WHAT DIDN'T

What mattered most. In order of impact: (1) fixing a missing deterministic routing rule for symbols (+10.7pt, Phase 1) — a reminder that pipeline bugs can dwarf modeling choices; (2) moving from a pretrained-but-not-fine-tuned LLM to a fine-tuned one, and further from beam-search scoring to teacher-forced candidate scoring; (3) adding a second, architecturally different LLM (Mistral) to the blend (+2.0pt) — diversity of errors mattered more than solo accuracy, evidenced by Mistral being weighted *above* Qwen in early blends despite scoring lower alone; (4) giving the number category its own model trained on genuinely new data (Gigaword), not just a narrower slice of the same corpus.

What didn't matter, or actively hurt. Ranking-tie-break logic (BaseScore vs. n-gram-order-first) moved accuracy by ~ 0.1 – 0.3 pp regardless of data or blend stage. Cleaning punctuation out of the training corpus looked principled but was actively harmful (-6.2 pp) because it mismatched the eval context's tokenization. Adding model complexity on top of an already-informative scalar blend — a learned GBDT reranker, or an embedding-similarity reranker — consistently made things worse, because the “new” signal wasn't actually independent of what the LM's own log-probability already encoded. Order-aware (rather than probability-weighted) cross-model interpolation lost to a fixed λ for the same underlying reason: n-gram order is not a trustworthy cross-model quality signal, but probability mass is.

VI. CONCLUSION

The submitted system for the test set is the full Phase-2 pipeline: deterministic symbol prediction, a number-specific Kneser–Ney 5-gram for the *number* category, and a three-way probability blend of a 6h-continued Qwen2.5-3B LoRA fine-tune, a zero-shot Mistral-7B-v0.1, and a general Kneser–Ney 5-gram for the *word* category. The final, full-scale, held-out result (§III-B6/Table VIII) is **78.60% overall / 76.09% word accuracy** (± 0.8 pt at **95% CI**, $n=9,999$), consistent with the earlier 77.68%/75.09% pooled-CV estimate from

the smaller ablation-stage sample (§IV-B). This was chosen over every simpler alternative tested (n-gram alone, either LLM alone, a 2-way blend, a learned reranker) because each of those was measured, under the identical evaluation protocol, to score lower — the gains are not assumed, they are the direct result of the ablations in §III-B and §5. All three models were trained/fine-tuned only on the official `train_final.src.tok` (and, for the number-specific KenLM, an additional Gigaword slice); no component was retrained on a larger corpus purely for the final submission beyond the already-described 6-hour LoRA continuation.

What worked well: treating the task as three independent sub-problems by answer category, rather than one monolithic model, since symbol and number have near-deterministic structure an LLM cannot exploit as directly as a targeted rule or a corpus-specific n-gram; and validating every design decision (scorer choice, candidate set, blend weights) under a consistent held-out protocol rather than a single noisy split, which caught at least one early result (the 72.50% single-split λ estimate) that would otherwise have been reported as real.

Challenges: the largest was procedural rather than algorithmic — a full project's worth of work (Phase 1) was built on data that turned out not to be the official contest files, discovered only after a from-scratch GPT-2, a full KenLM pipeline, and a Qwen2.5-0.5B fine-tune had already been run to completion. The architecture and smoothing lessons from that phase transferred directly to Phase 2 and were not wasted, but the compute and wall-clock time were. Secondary challenges were engineering rather than modeling: a naïve n-gram trainer that blew past a 14GB memory cap on the full Gigaword corpus before its first prune ever ran, and several non-obvious library-level bugs in the fine-tuned-model scoring path (an in-place-mutating KV-cache method that silently drops the cache when its return value isn't reassigned, and an inference-time patch that hard-asserted single-token decoding and rejected multi-candidate batches). The final full-scale run added two more: an over-broad error-grep that false-flagged a successful pipeline stage as failed, and a leftover smoke-test row limit that silently under-ran a scoring pass — both caught by a row-count assertion before reaching the submitted file, which is the reason that assertion is worth keeping in any pipeline with an unattended, multi-stage run.

Given more time or resources: the clearest next step is closing the remaining 9.5%-to-open candidate-set ceiling further by widening the beam again or generating candidates a different way, since the blend can only recover a word that is in its union candidate set; a close second is retraining the

TABLE VI
PROGRESSION TO THE FINAL SYSTEM — POOLED STRATIFIED 5-FOLD CV ON 9,448 ROWS (8,198 WORD / 1,048 SYMBOL / 202 NUMBER)

Stage	Word acc.	Overall
N-gram baseline (order-4)	55.51%	60.12%
Qwen2.5-3B zero-shot (word only)	70.26%	71.55%
Qwen2.5-3B LoRA fine-tuned, solo	69.72%	—
+ n-gram blend (raw-count scorer)	71.52%	74.47%
+ KenLM (KN5) scorer swap	71.74%	74.66%
+ teacher-forced candidate scoring	72.62%	75.42%
+ Mistral-7B 3-way blend	74.64%	77.29%*
+ number-specific KN5	—	77.29%†
+ 6h continued fine-tune, $k=10$ beam	75.09%	77.68%

*with number routed by the general n-gram; †the number-specific KN5 lifts number accuracy from 68.32% to 73.27% on this sample, moving overall from 77.18% to 77.29% before the final beam-widening step.

TABLE VII
FINAL PIPELINE BREAKDOWN (POOLED-CV SAMPLE, SUBMITTED SYSTEM)

Category	Method	Acc.
symbol	deterministic — predict the given first letter	98.76%
number	number-specific KenLM (train + Gigaword slice)	73.27%
word	3-way blend (Qwen LoRA + Mistral zero-shot + KN5)	75.09%
overall		77.68%

TABLE VIII
FINAL FULL-SCALE RESULT (SUBMITTED SYSTEM)

Cat.	Method	Acc.	n
symbol	deterministic first letter	99.10%	1109
number	number-specific KenLM	73.83%	214
word	3-way blend ($\lambda_q=0.40$, $\lambda_m=0.50$, $\lambda_{kn5}=0.10$)	76.09%	8676
overall		78.60% ± 0.80pt	9,999

Kneser–Ney n-gram at order 6/7 (validated as a small but real gain on Model A alone) and re-running the full 3-way blend on top of it, which was measured as promising but not yet combined with the final blend due to time.